

Programmation fonctionnelle 2
Preuves et programmes
CM8 : Plusieurs constructeurs et/ou plusieurs
arguments

Pierre Rousselin

Université Sorbonne Paris Nord

mars 2026

Arbres binaires

Entiers en binaire

Arbres binaires

- ▶ Définition avec les doigts :
 - ▶ une racine
 - ▶ chaque sommet a (au plus) deux enfants
 - ▶ les extrémités sont appelées feuilles
 - ▶ en informatique, en général, on met des données dans les sommets
- ▶ De nombreuses variations possibles, selon que :
 - ▶ on autorise ou pas l'arbre vide
 - ▶ on colore (arbres rouge noir) ou pas les sommets
 - ▶ etc

Arbres binaires stricts

- ▶ Comme premier exemple, on considère des arbres où l'arbre vide n'est pas autorisé.
- ▶ Un arbre est soit réduit à une seule feuille (contenant une donnée d'un certain type),
- ▶ soit sous la forme d'une racine (avec une donnée d'un certain type) avec un sous-arbre gauche et un sous-arbre droite.

Arbres binaires stricts

- ▶ Comme premier exemple, on considère des arbres où l'arbre vide n'est pas autorisé.
- ▶ Un arbre est soit réduit à une seule feuille (contenant une donnée d'un certain type),
- ▶ soit sous la forme d'une racine (avec une donnée d'un certain type) avec un sous-arbre gauche et un sous-arbre droite.

En OCaml :

```
type 'a sbtree = Leaf of 'a | Node of 'a sbtree * 'a * 'a sbtree
let exple1 =
  Node (Node (Leaf "bar", "plop", Leaf "baz"), "foo", Leaf "hop")
```

En Rocq

```
Inductive sbtree (A : Type) :=
  | leaf (key : A)
  | node (l : sbtree A) (key : A) (r : sbtree A).
Arguments leaf {A}. (* On rend A implicite. *)
Arguments node {A}.

From Stdlib Require Import String Ascii.
Open Scope string_scope.
Definition exple1 :=
  node (node (leaf "bar") "plop" (leaf "baz")) "foo" (leaf "hop").
```

Injectivité des constructeurs

On peut encore utiliser `injection`.

```
Lemma leaf_eq_leaf {A : Type} (k1 k2 : A) :  
  leaf k1 = leaf k2 -> k1 = k2.
```

```
Proof.
```

```
  intros H.  
  injection H.  
  exact H.
```

```
Qed.
```

```
Lemma node_eq_node {A : Type} (k1 k2 : A) (l1 l2 r1 r2 : sbtree A) :  
  node l1 k1 r1 = node l2 k2 r2 -> l1 = l2 /\ k1 = k2 /\ r1 = r2.
```

```
Proof.
```

```
  intros H.  
  injection H as Hl Hk Hr.  
  split; [| split]; assumption.
```

```
Qed.
```

discriminate

On peut encore utiliser `discriminate`.

```
Lemma leaf_not_node {A : Type} (k1 k2 : A) (l r : sbtree A) :  
  leaf k1 = node l k2 r -> 2 + 2 = 5.
```

`Proof.`

```
  intros H. discriminate H.
```

`Qed.`

Hauteur et nombre de sommets

- ▶ Hauteur (nombre de sommets maximal entre la racine et une des feuilles)

Hauteur et nombre de sommets

- Hauteur (nombre de sommets maximal entre la racine et une des feuilles)

```
Fixpoint height {A : Type} (t : sbtree A) :=  
  match t with  
  | leaf _ => 1  
  | node l _ r => S (max (height l) (height r))  
end.
```

- Nombre de sommets :

Hauteur et nombre de sommets

- Hauteur (nombre de sommets maximal entre la racine et une des feuilles)

```
Fixpoint height {A : Type} (t : sbtree A) :=  
  match t with  
  | leaf _ => 1  
  | node l _ r => S (max (height l) (height r))  
  end.
```

- Nombre de sommets :

```
Fixpoint numNodes {A : Type} (t : sbtree A) :=  
  match t with  
  | leaf _ => 1  
  | node l _ r => S (numNodes l + numNodes r)  
  end.
```

Nombre de feuilles et de sommets internes

- ▶ Nombre de feuilles

Nombre de feuilles et de sommets internes

- Nombre de feuilles

```
Fixpoint numLeaves {A : Type} (t : sbtree A) :=  
  match t with  
  | leaf _ => 1  
  | node l _ r => numLeaves l + numLeaves r  
end.
```

- Nombre de sommets internes

Nombre de feuilles et de sommets internes

- Nombre de feuilles

```
Fixpoint numLeaves {A : Type} (t : sbtree A) :=  
  match t with  
  | leaf _ => 1  
  | node l _ r => numLeaves l + numLeaves r  
  end.
```

- Nombre de sommets internes

```
Fixpoint numInnerNodes {A : Type} (t : sbtree A) :=  
  match t with  
  | leaf _ => 0  
  | node l _ r => S (numInnerNodes l + numInnerNodes r)  
  end.
```

- Que dire de la somme des deux fonctions précédentes ?

Preuve par induction sur les arbres binaires

Le constructeur `node` a deux paramètres de type `sbtree`, donc il y aura deux hypothèses d'induction.

Lemma `add_numInnerNodes_numLeaves_eq_numNodes {A : Type}`

`(t : sbtree A) : numInnerNodes t + numLeaves t = numNodes t.`

Proof.

`induction t as [| l IHl k r IHr].`

`- reflexivity.`

`- simpl.`

(Au secours ! *)*

`Set Printing Parentheses.`

`rewrite -add_assoc.`

`rewrite (add_assoc (numInnerNodes r)).`

`rewrite (add_comm (numInnerNodes r)).`

`rewrite !add_assoc.`

`rewrite IHl.`

`rewrite -add_assoc.`

`rewrite IHr.`

`reflexivity.`

Qed.

Parenthèse sur la tactique `lia`

- ▶ Il existe une tactique très puissante nommée `lia` pour *Linear Integer Arithmetic*.
- ▶ Elle sait résoudre des systèmes d'équations ou d'inéquations *linéaires* entre entiers.
- ▶ À n'utiliser que lorsque c'est explicitement autorisé
- ▶ et qu'on est déjà assez à l'aise sans automatisation
- ▶ ou que prouver des choses élémentaires sur les entiers n'est pas le sujet principal.

lia. Qed.

- Ici, ça donne la chose suivante :

```
From Stdlib Require Import Lia.
```

```
Lemma add_numInnerNodes_numLeaves_eq_numNodes' {A : Type}  
  (t : sbtree A) : numInnerNodes t + numLeaves t = numNodes t.
```

```
Proof.
```

```
  induction t as [| l IHl k r IHr].
```

```
  - reflexivity.
```

```
  - simpl. rewrite -IHl -IHr.
```

```
    lia.
```

```
Qed.
```

- Ou même, suivant l'humeur :

```
Tactic Notation "fait_pas_ch***" := lia.
```

```
Lemma add_numInnerNodes_numLeaves_eq_numNodes'' {A : Type}  
  (t : sbtree A) : numInnerNodes t + numLeaves t = numNodes t.
```

```
Proof.
```

```
  induction t as [| l IHl k r IHr].
```

```
  - reflexivity.
```

```
  - simpl. fait_pas_ch***.
```

```
Qed.
```

Arbres binaires (non stricts)

```
Inductive btree (A : Type) :=  
  | emptyTree  
  | node (left : btree A) (key : A) (right : btree A).  
Arguments emptyTree {A}.  
Arguments node {A}.  
Definition exple2 :=  
  node emptyTree "foo" (node emptyTree "baz" emptyTree).  
Definition exple3 :=  
  node (node emptyTree "baz" emptyTree) "foo" emptyTree.  
Attention, exple2 et exple3 sont bien différents!
```

Arbres binaires (non stricts)

```
Inductive btree (A : Type) :=
  | emptyTree
  | node (left : btree A) (key : A) (right : btree A).
Arguments emptyTree {A}.
Arguments node {A}.
Definition exple2 :=
  node emptyTree "foo" (node emptyTree "baz" emptyTree).
Definition exple3 :=
  node (node emptyTree "baz" emptyTree) "foo" emptyTree.
Attention, exple2 et exple3 sont bien différents !
Lemma exple2_ne_exple3 : exple2 <> exple3.
Proof.
  intros H.
  discriminate H.
Qed.
```

Arbres binaires

Entiers en binaire

Entiers strictement positifs en binaire

- Dans le prélude,

Inductive positive :=

```
| xI (p : positive) (* ajouter 1 à la fin *)  
| x0 (p : positive) (* ajouter 0 à la fin *)  
| xH. (* l'entier naturel 1 *)
```

- Intuitivement, `x0 p` est l'entier `p` avec un zéro en plus à la fin (donc multiplié par deux),
- `xI p` est l'entier `p` avec un « un » en plus à la fin (donc multiplié par deux, avec ajout de 1),
- On commence à `xH` qui représente 1. Pourquoi commencer à 1 ?
- Exemples :
 - `xI x0 xI xH` représente

Entiers strictement positifs en binaire

- Dans le prélude,

Inductive positive :=

```
| xI (p : positive) (* ajouter 1 à la fin *)  
| x0 (p : positive) (* ajouter 0 à la fin *)  
| xH. (* l'entier naturel 1 *)
```

- Intuitivement, `x0 p` est l'entier `p` avec un zéro en plus à la fin (donc multiplié par deux),
- `xI p` est l'entier `p` avec un « un » en plus à la fin (donc multiplié par deux, avec ajout de 1),
- On commence à `xH` qui représente 1. Pourquoi commencer à 1 ?
- Exemples :
 - `xI x0 xI xH` représente 13

Entiers strictement positifs en binaire

- ▶ Dans le prélude,

Inductive positive :=

```
| xI (p : positive) (* ajouter 1 à la fin *)  
| x0 (p : positive) (* ajouter 0 à la fin *)  
| xH. (* l'entier naturel 1 *)
```

- ▶ Intuitivement, `x0 p` est l'entier `p` avec un zéro en plus à la fin (donc multiplié par deux),
- ▶ `xI p` est l'entier `p` avec un « un » en plus à la fin (donc multiplié par deux, avec ajout de 1),
- ▶ On commence à `xH` qui représente 1. Pourquoi commencer à 1 ?
- ▶ Exemples :
 - ▶ `xI x0 xI xH` représente 13
 - ▶ `x0 xI x0 xH` représente

Entiers strictement positifs en binaire

- Dans le prélude,

```
Inductive positive :=  
  | xI (p : positive) (* ajouter 1 à la fin *)  
  | x0 (p : positive) (* ajouter 0 à la fin *)  
  | xH. (* l'entier naturel 1 *)
```

- Intuitivement, `x0 p` est l'entier `p` avec un zéro en plus à la fin (donc multiplié par deux),
- `xI p` est l'entier `p` avec un « un » en plus à la fin (donc multiplié par deux, avec ajout de 1),
- On commence à `xH` qui représente 1. Pourquoi commencer à 1 ?
- Exemples :
 - `xI x0 xI xH` représente 13
 - `x0 xI x0 xH` représente 10

Entiers strictement positifs en binaire

- Dans le prélude,

```
Inductive positive :=  
  | xI (p : positive) (* ajouter 1 à la fin *)  
  | x0 (p : positive) (* ajouter 0 à la fin *)  
  | xH. (* l'entier naturel 1 *)
```

- Intuitivement, `x0 p` est l'entier `p` avec un zéro en plus à la fin (donc multiplié par deux),
- `xI p` est l'entier `p` avec un « un » en plus à la fin (donc multiplié par deux, avec ajout de 1),
- On commence à `xH` qui représente 1. Pourquoi commencer à 1 ?
- Exemples :
 - `xI x0 xI xH` représente 13
 - `x0 xI x0 xH` représente 10
 - Attention, les chiffres sont « dans l'ordre inverse » de ce qu'on a l'habitude d'écrire.

Successesseur sur `positive`

À vous !

Successeur sur positive

À vous!

```
Fixpoint succ (p : positive) :=  
  match p with  
  | xH => x0 xH  
  | x0 q => xI q  
  | xI q => x0 (succ q)  
end.
```

Addition sur `positive`

À vous (souvenirs d'école primaire, mais en binaire donc plus simple) !

Addition sur positive

À vous (souvenirs d'école primaire, mais en binaire donc plus simple)!

```
Fixpoint add (p q : positive) :=  
  match p with  
  | xH => match q with  
          | xH => x0 xH  
          | x0 q' => xI q'  
          | xI q' => x0 (succ q')  
        end  
  | x0 p' => match q with  
              | xH => xI p'  
              | x0 q' => x0 (add p' q')  
              | xI q' => xI (add p' q')  
            end  
  | xI p' => match q with  
              | xH => x0 (succ p')  
              | x0 q' => xI (add p' q')  
              | xI q' => x0 (succ (add p' q'))  
            end  
  end.
```

Commutativité de l'addition

Lemma add_comm (p q : positive) : p + q = q + p.

Proof.

```
induction p as [p' IH | p' IH []] in q |- *.
- destruct q as [q' | q' []].
  + simpl. rewrite IH. reflexivity.
  + simpl. rewrite IH. reflexivity.
  + simpl. reflexivity.
- destruct q as [q' | q' []].
  + simpl. rewrite IH. reflexivity.
  + simpl. rewrite IH. reflexivity.
  + simpl. reflexivity.
- destruct q; reflexivity.
```

Qed.

Remarquer :

```
induction p as [p' IH | p' IH []] in q |- *.
```

pour généraliser l'hypothèse d'induction à tous les `positive q`.

Commutativité de l'addition, version courte

Toutes les sous-preuves sont les mêmes, sauf dans 4 cas sur les 9 où on `rewrite IH` entre `simpl` et `reflexivity`. La « tacticielle » `try` demande à Rocq d'appliquer une tactique uniquement dans le cas où elle réussit. Ici c'est pratique, mais attention, en général ce n'est pas très robuste (peut « cacher » une erreur, qui apparaîtra alors bien plus tard).

Lemma `add_comm'` (`p q` : `positive`) : `p + q = q + p`.

Proof.

```
induction p as [? IH | ? IH |] in q |- *;  
  destruct q; simpl; try rewrite IH; reflexivity.
```

Qed.